

Project 1: NVIDIA (NVDA) Stock Analysis

Jooho Kim

2025-10-28

1. Background and Problem Definition

NVIDIA (NVDA), a central player in the AI boom, has seen dynamic but highly volatile stock price growth, especially post-2020. This volatility makes forecasting challenging, as the price is driven not only by historical patterns but also by external market sentiment. Factors like macroeconomic news and political statements, often quantified by the CBOE Volatility Index (VIX) or “fear index,” significantly impact its price.

Traditional univariate models like ARIMA are ill-equipped for this environment as they cannot incorporate these external variables. Consequently, this project adopts a machine learning approach. As noted by Machine Learning Mastery (2021), ensemble methods like XGBoost can effectively model time-series by reframing the task as a supervised learning problem.

The objective is to develop a short-term XGBoost forecasting model for NVDA. We will test the hypothesis that a model enriched with technical indicators and market sentiment data (VIX) can yield more accurate predictions than models relying on price data alone.

The analysis is divided into three main tasks:

- **Data Acquisition & Cleaning:** Sourcing NVDA (OHLCV) and VIX data; engineering a feature set of technical indicators, lags, and sentiment.
- **Exploratory Data Analysis (EDA):** Visualizing historical trends, volatility, and the NVDA-VIX relationship.
- **XGBoost Modeling:** Implementing, tuning, and evaluating the short-term forecast model.

(Reference: Brownlee, J. (2021). “How to Use XGBoost for Time Series Forecasting.” Machine Learning Mastery. Retrieved from <https://machinelearningmastery.com/xgboost-for-time-series-forecasting/>)

2. Data Wrangling and Cleaning

This section covers loading the raw data, cleaning it, and transforming it into a structured format suitable for analysis.

2.1. Load Libraries

First, we load all necessary R packages for data processing, analysis, and visualization.

```
library(httr)      # For making API calls
library(jsonlite)  # For parsing JSON data
library(dplyr)     # For data manipulation
library(readr)     # For reading/writing CSV files
library(ggplot2)   # For plotting
library(TTR)       # For technical indicators (SMA, RSI, MACD)
library(xgboost)   # For XGBoost modeling
library(zoo)       # For rollapply
```

2.2. Data Acquisition (API Call)

We acquire two primary datasets for this analysis.

- **NVIDIA (NVDA) Price Data:** We use `httr` to call the Alpha Vantage API for NVDA's daily prices. The raw JSON is saved to `data/` to avoid repeated API calls.
- **Market Sentiment Data:** We will also load historical data for the CBOE Volatility Index (VIX), which serves as our "fear index."

Both datasets will later be processed and joined by their common date column to create the final feature set for modeling.

```
api_key <- "" # Key is removed for privacy
symbol <- "NVDA"
api_url <- paste0("https://www.alphavantage.co/query?function=TIME_SERIES_DAILY",
                  "&symbol=", symbol, "&outputsize=full", "&apikey=", api_key)

response <- GET(api_url)
json_content <- content(response, "text", encoding = "UTF-8")

raw_json_path <- "data/raw_nvda_data.json"
write_lines(json_content, file = raw_json_path)
```

2.3. Data Cleaning and Processing

This section involves processing both the NVDA and VIX datasets and then combining them.

First, the raw, nested JSON for NVDA is parsed, converted into a data frame, and cleaned. Columns are renamed for clarity (e.g., `X1.open` to `open`), and all values are converted to their proper data types (Date, Numeric).

Second, the downloaded VIX (CBOE Volatility Index) data is loaded from its CSV file. We clean this dataset by renaming its columns, filtering out non-trading days (where the value is `.`), and converting its values to Date and Numeric types.

Finally, we perform an inner join on the two datasets using their common date column. This operation aligns both time series and automatically discards any dates where either market was closed (e.g., weekends, holidays). This results in a single, clean, and synchronized dataset ready for analysis.

```
# 1. Load and Process NVDA JSON
raw_data <- fromJSON("data/raw_nvda_data.json")
time_series_data <- raw_data$`Time Series (Daily)`
```

```

df <- data.frame(
  date = names(time_series_data),
  do.call(rbind, time_series_data),
  row.names = NULL
)

# Rename columns
df_cleaned <- df %>%
  rename(
    open = `X1..open`,
    high = `X2..high`,
    low = `X3..low`,
    close = `X4..close`,
    volume = `X5..volume`
  )

# Convert data types and sort NVDA data
df_nvda <- df_cleaned %>%
  mutate(
    date = as.Date(date),
    open = as.numeric(open),
    high = as.numeric(high),
    low = as.numeric(low),
    close = as.numeric(close),
    volume = as.numeric(volume)
  ) %>%
  arrange(date)

# 2. Load and Process VIX Data
# Data sourced from FRED: https://fred.stlouisfed.org/series/VIXCLS
# Manually downloaded and saved as 'data/VIXCLS.csv'
vix_data <- read_csv("data/VIXCLS.csv")

vix_cleaned <- vix_data %>%
  rename(date = observation_date, vix_close = VIXCLS) %>%
  filter(vix_close != ".") %>% # Filter out non-trading days (marked as ".")
  mutate(
    date = as.Date(date),
    vix_close = as.numeric(vix_close)
  )

# 3. Join Datasets
# An inner join synchronizes the datasets, keeping only dates
# where both the stock market and VIX data are available.
df_final <- inner_join(df_nvda, vix_cleaned, by = "date")

# Save the final joined dataset
write_csv(df_final, "data/processed_joined_data.csv")

# Display summary
cat("Final joined dataset dimensions:", nrow(df_final), "rows and", ncol(df_final), "columns\n")

## Final joined dataset dimensions: 1463 rows and 7 columns

```

```
head(df_final)
```

```
##      date  open  high  low  close  volume  vix_close
## 1 2020-01-02 238.75 239.91 236.72 239.91 5941969    12.47
## 2 2020-01-03 235.10 237.83 234.10 236.07 5144308    14.02
## 3 2020-01-06 232.32 237.27 231.27 237.06 6572812    13.85
## 4 2020-01-07 238.20 241.77 236.39 239.93 7980144    13.79
## 5 2020-01-08 239.76 242.04 238.15 240.38 6931017    13.45
## 6 2020-01-09 243.85 245.93 240.86 243.02 6386053    12.54
```

3. Exploratory Data Analysis (EDA)

Before modeling, we explore the combined `df_final` dataset. The goal is to understand historical trends, volatility patterns, and the key relationships between NVIDIA's stock price and overall market sentiment (VIX).

3.1. Basic Summary Statistics

We first examine the summary statistics for the entire joined dataset, which now includes the `vix_close` column.

```
summary(df_final)
```

```
##      date      open      high      low
## Min.   :2020-01-02  Min.   : 87.46  Min.   : 99.44  Min.   : 86.62
## 1st Qu.:2021-06-15  1st Qu.: 161.07  1st Qu.: 165.44  1st Qu.: 158.53
## Median :2022-11-25  Median : 253.80  Median : 259.50  Median : 247.84
## Mean   :2022-11-27  Mean   : 333.19  Mean   : 339.15  Mean   : 326.89
## 3rd Qu.:2024-05-11  3rd Qu.: 479.62  3rd Qu.: 487.16  3rd Qu.: 472.76
## Max.   :2025-10-27  Max.   :1240.48  Max.   :1255.87  Max.   :1183.20
##      close      volume      vix_close
## Min.   : 94.31  Min.   : 2447088  Min.   :11.86
## 1st Qu.: 162.40  1st Qu.: 18498720  1st Qu.:15.96
## Median : 252.73  Median : 45441366  Median :19.15
## Mean   : 333.41  Mean   : 85943691  Mean   :21.11
## 3rd Qu.: 480.43  3rd Qu.: 84932478  3rd Qu.:24.30
## Max.   :1224.40  Max.   :818830895  Max.   :82.69
```

The `summary()` output shows our final dataset spans from January 2, 2020 to October 27, 2025. This period, which starts just before the 2020 COVID-19 crash, was selected by the intersection of our available NVDA and VIX data.

- **NVDA Close:** The closing price has ranged from a minimum of \$94.31 to a maximum of \$1224.40. (Note: NVIDIA executed a 10-for-1 stock split in June 2024, so historical prices reflect pre-split values).
- **VIX Close:** Our market sentiment feature, the VIX, has ranged from a low of \$11.86 (indicating market stability) to a high of \$82.69. This high value reflects the extreme market panic during the 2020 crash, confirming our dataset includes periods of extreme sentiment.

3.2. Handling Stock Split Adjustment

NVIDIA executed a 10-for-1 stock split on June 10, 2024. This creates a structural break in the price data that would distort our time-series analysis and subsequent XGBoost modeling. To ensure continuity, we adjust all pre-split prices by dividing them by 10, making all historical prices comparable to current post-split values.

```
# Define the split date
split_date <- as.Date("2024-06-10")

# Adjust pre-split prices
df_adjusted <- df_final %>%
  mutate(
    open = ifelse(date < split_date, open / 10, open),
    high = ifelse(date < split_date, high / 10, high),
    low = ifelse(date < split_date, low / 10, low),
    close = ifelse(date < split_date, close / 10, close)
  )

# Display adjusted summary
cat("Adjusted Summary:\n")
```

Adjusted Summary:

```
summary(df_adjusted$close)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  11.23   24.72   46.77   63.01   92.61  192.57
```

After adjustment, all prices are now on a consistent scale, allowing for accurate feature engineering and modeling.

3.3. Historical Price Trend

We visualize the adjusted closing price over time to observe long-term trends and major price movements within our selected 2020-2025 timeframe.

```
ggplot(df_adjusted, aes(x = date, y = close)) +
  geom_line(color = "steelblue") +
  scale_x_date(breaks = seq(as.Date("2020-01-01"), as.Date("2026-01-01"), by = "1 year"),
    date_labels = "%Y") +
  labs(title = "NVIDIA (NVDA) Adjusted Historical Closing Price (2020-2025)",
    x = "Date",
    y = "Adjusted Closing Price (USD)") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



The plot starkly captures the volatility of the post-2020 era. We can clearly see:

- The sharp, brief crash during the COVID-19 pandemic in early 2020.
- A subsequent explosive growth phase through 2021.
- A major correction and bear market throughout 2022.
- A rapid recovery and unprecedented bull run from 2023 onward, driven by the generative AI boom, leading to new all-time highs in 2024-2025.

3.4. Volatility Analysis

To better understand the sharp price corrections observed in the previous plot, we calculate the 30-day rolling volatility based on daily returns. This measures risk and price instability over time, helping us identify periods of extreme market stress.

```
# Calculate daily returns
df_adjusted <- df_adjusted %>%
  mutate(daily_return = (close - lag(close)) / lag(close))

# Calculate 30-day rolling volatility
df_adjusted <- df_adjusted %>%
  mutate(volatility_30d = zoo::rollapply(daily_return,
                                         width = 30,
                                         FUN = sd,
                                         fill = NA,
                                         align = "right"))

# Plot volatility
ggplot(df_adjusted, aes(x = date, y = volatility_30d)) +
  geom_line(color = "darkred") +
  scale_x_date(breaks = c(seq(as.Date("2000-01-01"), as.Date("2015-01-01"), by = "2 years"),
                           seq(as.Date("2016-01-01"), as.Date("2026-01-01"), by = "1 year"))),
```

```

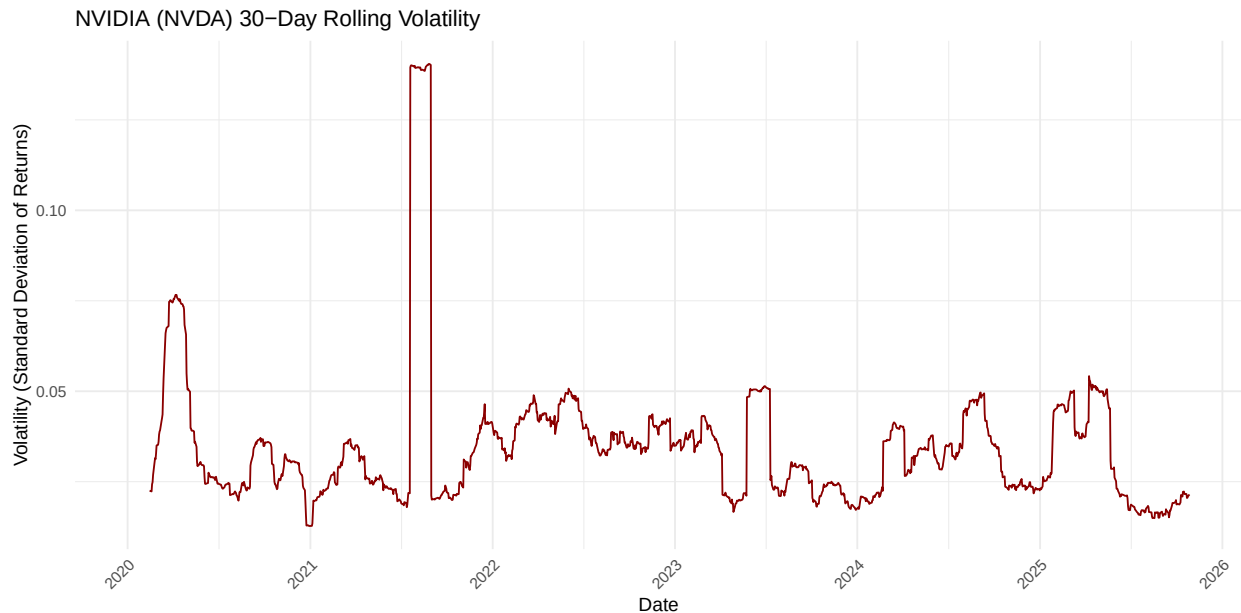
date_labels = "%Y") +
labs(title = "NVIDIA (NVDA) 30-Day Rolling Volatility",
     x = "Date",
     y = "Volatility (Standard Deviation of Returns)") +
theme_minimal() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))

```

```

## Warning: Removed 30 rows containing missing values or values outside the scale range
## ('geom_line()').

```



The volatility plot clearly confirms the dynamic nature of the stock within our 2020-2025 timeframe. The analysis highlights several key periods of extreme stress:

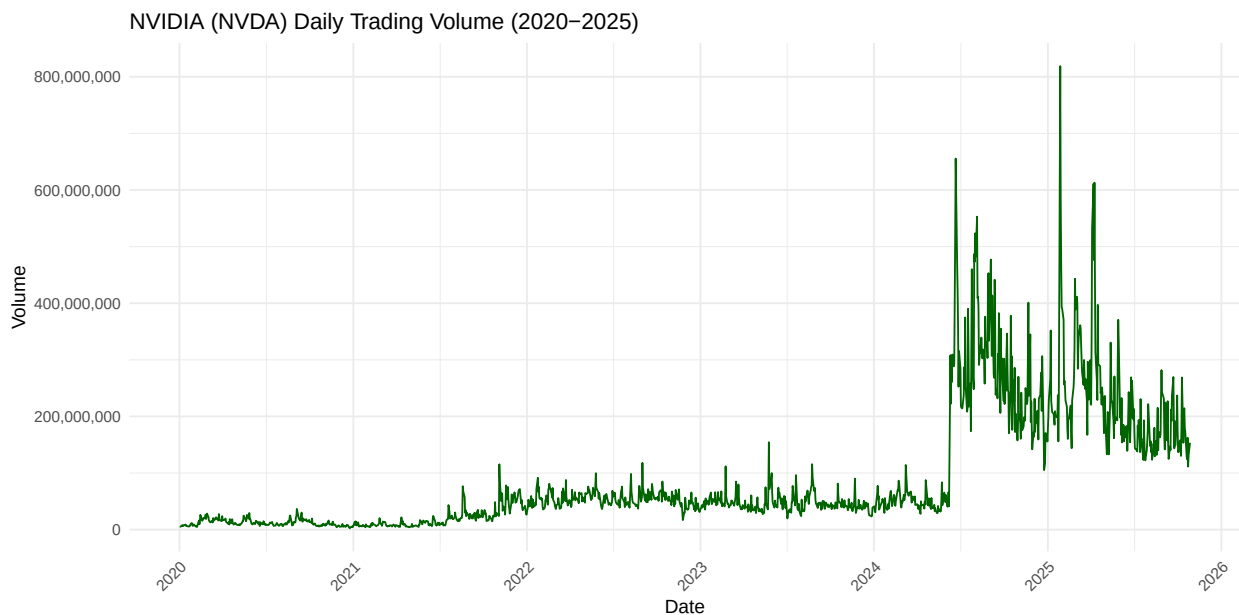
- **Early 2020:** A major spike in volatility is visible, coinciding with the COVID-19 market crash.
- **Late 2021 - Early 2022:** The most dramatic spike in the entire dataset occurs here. This corresponds perfectly to the severe tech sector correction and bear market of 2022.
- **2023-2025 (AI Boom):** Even during the explosive bull run of the AI boom, the plot shows that volatility did not return to low levels. It remained consistently elevated with frequent, sharp spikes, indicating continuous high risk and investor uncertainty.

This analysis of NVIDIA's inherent volatility strongly supports our decision to include market-wide volatility (the `vix_close` feature) as a key predictor in our model.

3.5. Trading Volume Analysis

Trading volume provides insight into market activity and investor interest, and it is a key feature in our XGBoost model. High volume during price movements can indicate strong conviction behind the trend.

```
ggplot(df_adjusted, aes(x = date, y = volume)) +
  geom_line(color = "darkgreen") +
  # Adjust x-axis breaks to match our 2020-2025 data range
  scale_x_date(breaks = seq(as.Date("2020-01-01"), as.Date("2026-01-01"), by = "1 year"),
    date_labels = "%Y") +
  scale_y_continuous(labels = scales::comma) +
  labs(title = "NVIDIA (NVDA) Daily Trading Volume (2020-2025)",
    x = "Date",
    y = "Volume") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



The plot shows that trading volume remained relatively moderate and stable from 2020 through 2022. A dramatic surge began in 2023-2024, coinciding with the AI boom and NVIDIA's explosive stock price growth.

Peak volumes (confirmed by our summary data to exceed 800 million shares) occurred in 2024, reflecting unprecedented investor interest and market liquidity. This confirms that volume is a highly dynamic feature and justifies its inclusion in our model.

3.6. Price vs Market Sentiment(VIX)

```
library(patchwork) # For combining plots
```

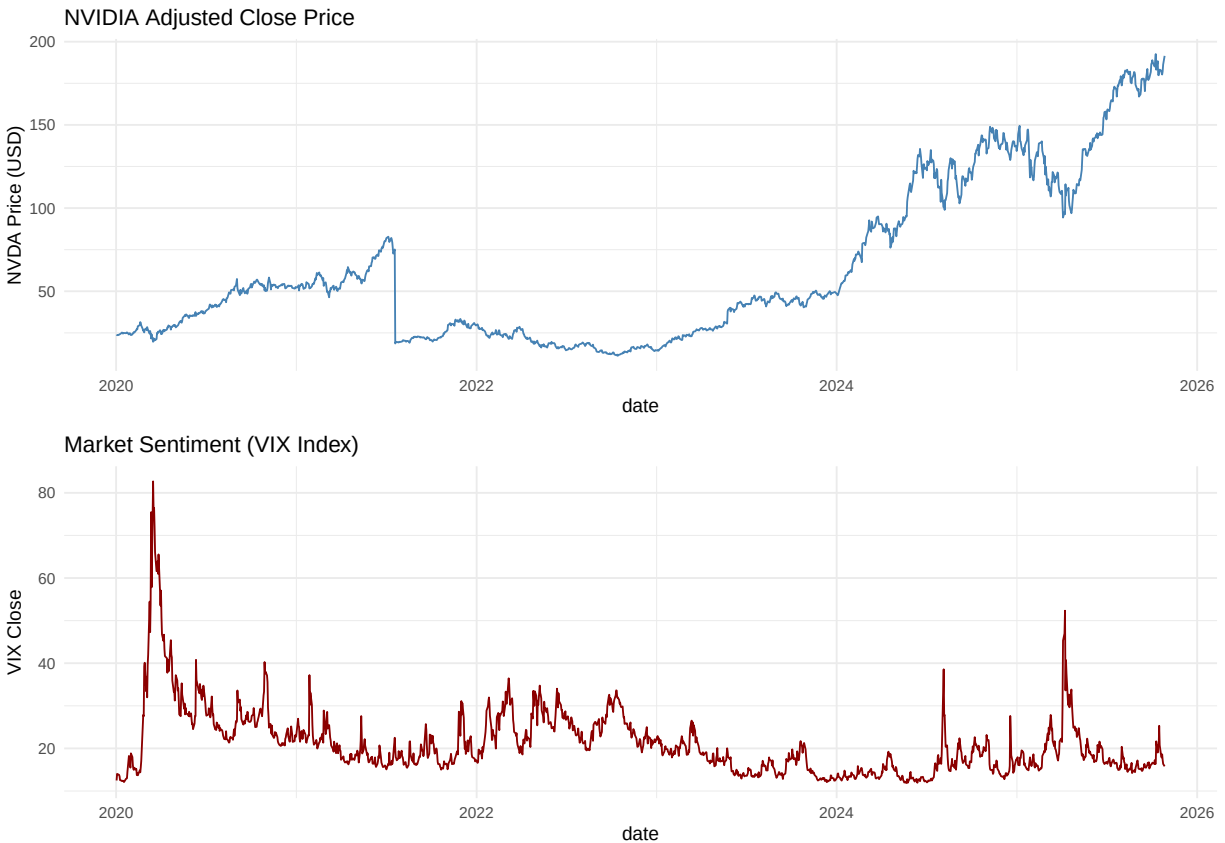
```
## Warning: package 'patchwork' was built under R version 4.3.3
```

```
# Plot 1: NVDA Close Price
p1 <- ggplot(df_adjusted, aes(x = date, y = close)) +
  geom_line(color = "steelblue") +
  labs(title = "NVIDIA Adjusted Close Price", y = "NVDA Price (USD)") +
  theme_minimal()
```



```
# Plot 2: VIX Close Price
p2 <- ggplot(df_adjusted, aes(x = date, y = vix_close)) +
  geom_line(color = "darkred") +
  labs(title = "Market Sentiment (VIX Index)", y = "VIX Close") +
  theme_minimal()

# Combine the two plots vertically, aligning the x-axis
p1 + p2 + plot_layout(ncol = 1)
```



The plots reveal a clear inverse correlation, but with an important nuance.

While the single largest VIX spike (the 80+ spike during the COVID-19 crash in early 2020) corresponds to only a moderate price dip for NVIDIA, the biggest NVIDIA price drop (the 2022 bear market) aligns perfectly with the sustained period of high VIX levels (hovering between 20-35) throughout 2022.

Furthermore, the massive bull run for NVIDIA from 2023-2025 corresponds to a period of sustained low VIX, indicating market stability. This visual evidence strongly supports our hypothesis that `vix_close` is a powerful (and complex) negative predictor for NVDA's price.

4. XGBoost Modeling

We now move from exploratory analysis to predictive modeling. Unlike ARIMA, which models data as a sequence, XGBoost requires a supervised learning setup. Our first step is Feature Engineering, where we transform our time-series data into a feature matrix (X) and a target vector (y).

4.1. Feature Engineering

Based on our project plan, we will create 18 features (X) to predict our target (y), which we define as the closing price 7 days into the future ($\text{Close}_{\{t+7\}}$).

We will use the TTR library to generate technical indicators and dplyr to create lag features.

```
library(TTR)

# Use df_adjusted from our EDA
df_model_data <- df_adjusted

# 1. Create Technical Indicators (SMA, RSI, MACD, BBands)
df_model_data <- df_model_data %>%
  mutate(
    SMA5 = SMA(close, n = 5),
    SMA20 = SMA(close, n = 20),
    RSI = RSI(close, n = 14),
    MACD = MACD(close, nFast = 12, nSlow = 26, nSig = 9)[, "macd"],
    BB_Upper = BBands(close, n = 20, sd = 2)[, "up"],
    BB_Lower = BBands(close, n = 20, sd = 2)[, "dn"]
  )

# 2. Create Lag Features (Past closing prices)
df_model_data <- df_model_data %>%
  mutate(
    Lag_Close_1 = lag(close, n = 1),
    Lag_Close_2 = lag(close, n = 2),
    Lag_Close_3 = lag(close, n = 3),
    Lag_Close_4 = lag(close, n = 4),
    Lag_Close_5 = lag(close, n = 5)
  )

# 3. Create Other Derived Features
df_model_data <- df_model_data %>%
  mutate(
    High_Low_Diff = high - low,
    Close_Open_Diff = close - open
  )

# 4. Create the Target Variable (y)
# We want to predict the price 7 days in the future
df_model_data <- df_model_data %>%
  mutate(
    Target_Close_t_plus_7 = lead(close, n = 7)
  )

# 5. Clean Data
# Remove all rows with NA values
# This removes:
# - The first ~30 rows (due to SMA20, MACD, RSI calculations)
# - The LAST 7 rows (because their Target_Close_t_plus_7 is NA)
df_clean <- na.omit(df_model_data)
```

```
# Display final dataset for modeling
cat("Final model dataset dimensions:", nrow(df_clean), "rows and", ncol(df_clean), "columns\n")
```

```
## Final model dataset dimensions: 1426 rows and 23 columns
```

```
head(df_clean)
```

```
##      date    open    high    low  close  volume  vix_close  daily_return
## 31 2020-02-14 28.726 29.497 28.600 28.979 26048093    13.68    0.07020459
## 32 2020-02-18 28.564 29.731 28.486 29.657 15547025    14.83    0.02339625
## 33 2020-02-19 30.149 31.541 30.149 31.470 17358281    14.38    0.06113228
## 34 2020-02-20 31.277 31.632 29.635 30.870 20251259    15.56   -0.01906578
## 35 2020-02-21 30.419 30.520 29.066 29.407 19232331    17.08   -0.04739229
## 36 2020-02-24 27.019 28.187 26.800 27.328 21422873    25.03   -0.07069745
##      volatility_30d  SMA5    SMA20    RSI    MACD  BB_Upper  BB_Lower
## 31      0.02252117 27.2794 25.37325 76.86150 3.286869 27.89935 22.84715
## 32      0.02230778 27.9514 25.60970 78.96367 4.048348 28.73828 22.48112
## 33      0.02431582 28.8876 25.94350 83.32606 5.107470 29.95322 21.93378
## 34      0.02487221 29.6108 26.23645 77.59142 5.672368 30.75472 21.71818
## 35      0.02688551 30.0766 26.44250 65.71542 5.617767 31.14088 21.74412
## 36      0.03038559 29.7464 26.55650 53.24386 4.918394 31.22455 21.88845
##      Lag_Close_1 Lag_Close_2 Lag_Close_3 Lag_Close_4 Lag_Close_5 High_Low_Diff
## 31      27.078      27.254      26.789      26.297      25.159      0.897
## 32      28.979      27.078      27.254      26.789      26.297      1.245
## 33      29.657      28.979      27.078      27.254      26.789      1.392
## 34      31.470      29.657      28.979      27.078      27.254      1.997
## 35      30.870      31.470      29.657      28.979      27.078      1.454
## 36      29.407      30.870      31.470      29.657      28.979      1.387
##      Close_Open_Diff Target_Close_t_plus_7
## 31          0.253          26.765
## 32          1.093          25.260
## 33          1.321          27.007
## 34         -0.407          27.643
## 35         -1.012          26.589
## 36          0.309          28.451
```

This code chunk creates our 16 engineered predictors, providing a comprehensive view of the stock's trend, momentum, volatility, and market sentiment.

- **Market Sentiment & Volatility:**

- **vix_close:** The “Fear Index,” a key external indicator of market sentiment.
- **daily_return:** The daily percentage change, normalizing price movements.
- **volatility_30d:** NVIDIA’s own 30-day historical risk, indicating its stability.
- **High_Low_Diff:** The daily price range, a direct measure of daily volatility.

- **Trend & Momentum Indicators:**

- **SMA5, SMA20:** 5-day and 20-day moving averages to capture short and medium-term trends.
- **RSI:** The Relative Strength Index, used to identify overbought (>70) or oversold (<30) conditions.
- **MACD:** A classic momentum indicator showing the relationship between two moving averages.

- `BB_Upper`, `BB_Lower`: Bollinger Bands, which identify statistically “extreme” price levels relative to the trend.
- `Close_Open_Diff`: The “candlestick body,” measuring the strength of the day’s price movement.
- **Lag (Memory) Features:**
 - `Lag_Close_1` to `Lag_Close_5`: The closing prices from the previous 5 days, giving the model a short-term “memory” of recent price action.

Finally, `na.omit()` cleans the dataset by removing the initial rows used for indicator warm-up and the final 7 rows that lack a future target value. The `df_clean` dataframe is now ready for modeling.